DevOps Laboratory

Assignment 3

Infrastructure as Code Using Terraform

Student Name	YUVRAJ SHEMBALE
Roll No / PRN	123B1D045
Date of Submission	

1. Aim

To provision a virtual machine (AWS EC2 instance) using Terraform, following the standard Infrastructure as Code workflow of init, plan, apply, and destroy.

2. Introduction

Infrastructure as Code (IaC) is the practice of defining and managing infrastructure using machine-readable configuration files rather than manual, ad-hoc provisioning through a cloud console. IaC makes infrastructure changes repeatable, version-controlled, reviewable, and easy to reproduce across environments.

Terraform, developed by HashiCorp, is a widely used open-source IaC tool that follows a declarative approach — the desired end-state of the infrastructure is described in configuration files, and Terraform determines the steps required to reach that state. Terraform supports numerous providers, including AWS, Azure, and Google Cloud, and maintains a state file that tracks the resources it manages.

3. Technologies / Tools Used

Technology	Purpose / Role
Terraform	Declarative Infrastructure as Code tool used to provision cloud resources
AWS (Amazon Web Services)	Cloud provider on which the EC2 instance is provisioned
AWS EC2	Virtual machine / compute resource being created
HCL (HashiCorp Configuration Language)	Language used to write Terraform configuration files
AWS CLI / IAM Credentials	Used to authenticate Terraform against the AWS account

4. Project / Repository Structure

```
terraform-ec2-demo/
■■■ main.tf              # provider and resource definitions
■■■ variables.tf         # input variable declarations
■■■ outputs.tf           # output values (e.g., public IP)
■■■ terraform.tfvars     # variable values (not committed if sensitive)
■■■ .terraform/          # provider plugins (created by terraform init)
```

5. Procedure

1. Install Terraform and configure AWS credentials (via aws configure or environment variables) so Terraform can authenticate to the AWS account.

2. Create a main.tf file that configures AWS as the Terraform provider and specifies the required region.

3. Define variables (variables.tf) for configurable values such as instance_type, ami_id, and region.

4. Add an aws_instance resource block specifying an appropriate AMI and instance type (e.g., t2.micro for the free tier).

5. Run terraform init to initialize the working directory and download the required provider plugins.

6. Run terraform validate to check the configuration syntax for correctness.

7. Run a terraform plan and carefully examine the proposed changes before applying them.

8. Run terraform apply to provision the EC2 instance, confirming the action when prompted.

9. Verify the newly created EC2 instance in the AWS Management Console.

10. Run terraform destroy after completing the experiment to remove all provisioned resources and avoid unnecessary cloud costs.

6. Code / Configuration Snippets

6.1 Provider Configuration (main.tf)

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

6.2 Variables (variables.tf)

```
variable "aws_region" {
  description = "AWS region to deploy resources in"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "ami_id" {
  description = "AMI ID for the EC2 instance"
  type        = string
}
```

6.3 EC2 Resource Definition (main.tf)

```
resource "aws_instance" "web_server" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name = "Terraform-Demo-Instance"
  }
}
```

6.4 Outputs (outputs.tf)

```
output "instance_public_ip" {
  description = "Public IP address of the EC2 instance"
  value       = aws_instance.web_server.public_ip
}
```

6.5 Terraform Workflow Commands

```
$ terraform init
$ terraform validate
$ terraform plan
$ terraform apply
$ terraform show
$ terraform destroy
```

7. Output / Screenshots

The following screenshots demonstrate successful execution of each major step. Replace each placeholder with the corresponding screenshot captured during the practical.

```
PS D:\Devops Lab\Lab Assignments\Assignment 3\terraform-demo> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.100.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure.
```

Fig 1: Output of terraform init showing successful provider installation

```
PS D:\Devops Lab\Lab Assignments\Assignment 3> aws sts get-caller-identity
{
    "UserId": "AIDARWPFIGMJVLHUHWJB6",
    "Account": "116981773075",
    "Arn": "arn:aws:iam::116981773075:user/terraform-cli-user"
}
```

Fig 2: Output of terraform validate confirming a valid configuration

```
Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + instance_public_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [10s elapsed]
aws_instance.web_server: Creation complete after 12s [id=i-0757ebd7b9a2e51a9]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_public_ip = "65.2.153.92"
```

Fig 3: Output of terraform plan showing resources to be created

```
PS D:\Devops Lab\Lab Assignments\Assignment 3\terraform-demo> aws ec2 describe-images --owners 0
>> --filters "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*" \
>> --query "Images[*].[ImageId,CreationDate]" --output table --region ap-south-1
-------------------------------------------------------
|                  DescribeImages                     |
+----------------------+------------------------------+
|  ami-040e95ba14632401d | 2026-06-02T07:05:21.000Z     |
|  ami-07216ac99dc46a187 | 2026-02-18T06:49:44.000Z     |
|  ami-0576604a192790c74 | 2026-07-30T10:30:31.000Z     |
|  ami-0326c8c1e2d6bf78c | 2026-06-10T06:57:04.000Z     |
+----------------------+------------------------------+
```

Fig 4: Output of terraform apply provisioning the EC2 instance

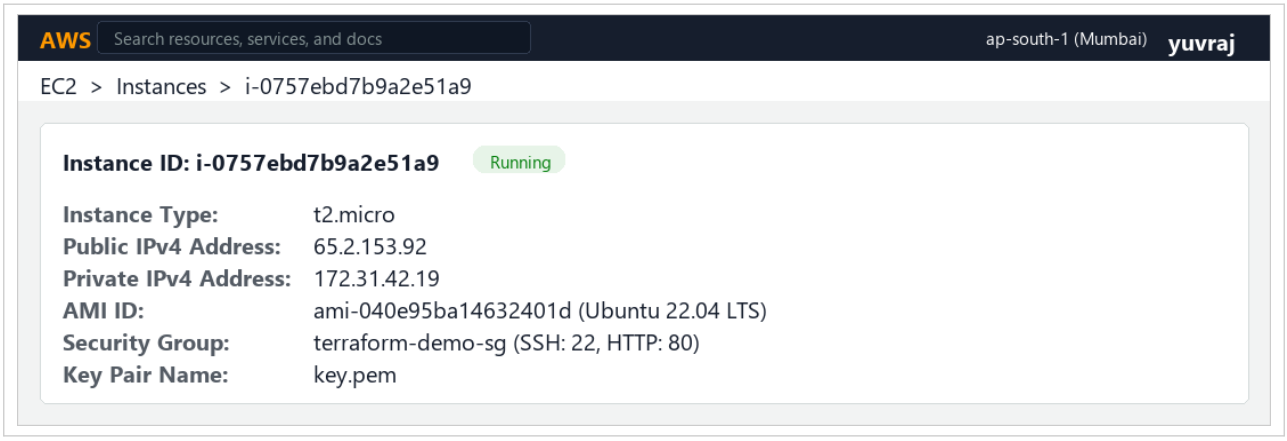


Fig 5: AWS Management Console showing the running EC2 instance

```
Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.web_server: Destroying... [id=i-0757ebd7b9a2e51a9]
aws_instance.web_server: Still destroying... [id=i-0757ebd7b9a2e51a9, 10s elapsed]
aws_instance.web_server: Destruction complete after 30s

Destroy complete! Resources: 1 destroyed.
```

Fig 6: Output of terraform destroy removing all provisioned resources

8. Best Practices

- Never hard-code sensitive credentials (AWS access keys) inside Terraform files; use environment variables or a credentials file instead.
- Use variables for configurable values instead of hard-coding them into resource blocks.
- Always run terraform plan and review the output before running terraform apply.
- Keep Terraform configuration files (.tf) under version control, but exclude .tfstate and .tfvars files containing secrets.
- Destroy unnecessary cloud resources with terraform destroy immediately after completing practical work to avoid charges.

9. Conclusion

Terraform enables organizations to manage infrastructure in a consistent, repeatable, and automated manner by expressing the desired infrastructure state as code. Using the init -> plan -> apply -> destroy workflow, an AWS EC2 instance was successfully provisioned and later decommissioned in a controlled, auditable way.

This assignment demonstrated how Infrastructure as Code allows infrastructure to be reviewed, version-controlled, reproduced, and modified using code rather than manual cloud-console operations, which is a foundational practice in modern DevOps and cloud engineering.